Leadership & Excellence [LOGO]	**SMART PORT & AIRPORT BAGGAGE TRACKING SYSTEM**
JAVA PROJECT **NOVEMBER 2026**	JAVA PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF **BACHELOR OF ENGINEERING** IN **COMPUTER SCIENCE AND ENGINEERING** OF THE ANNA UNIVERSITY
PROJECT WORK	Submitted by **RAGAV C S - 722825104179**
BATCH **2025 – 2029**	Under the Guidance of **MR. M. KARTHICK RAJA M.E.** Assistant Professor/CSE

Department of Computer Science & Engineering

SRI ESHWAR COLLEGE OF ENGINEERING

(An Autonomous Institution – Affiliated to Anna University)

COIMBATORE – 641 202

Sri Eshwar College of Engineering

(An Autonomous Institution – Affiliated to Anna University)

COIMBATORE – 641 202

BONAFIDE CERTIFICATE

Certified that this Report titled **"SMART PORT & AIRPORT BAGGAGE TRACKING SYSTEM"** is the bonafide work of

RAGAV C S	722825104179

who carried out the project work under my supervision.

SIGNATURE

DR. R. SUBHA M.E., Ph.D.
HEAD OF THE DEPARTMENT
Computer Science and Engineering,
Sri Eshwar College of Engineering,
Coimbatore – 641 202.

SIGNATURE

MR. M. KARTHICK RAJA M.E.
SUPERVISOR
Assistant Professor,
Computer Science and Engineering,
Sri Eshwar College of Engineering,
Coimbatore – 641 202.

Submitted for the **Autonomous Semester End Mini Project Viva-Voce** held on .....

_____	_____
INTERNAL EXAMINER	EXTERNAL EXAMINER

DECLARATION

RAGAV C S [722825104179]

To declare that the project entitled **"SMART PORT & AIRPORT BAGGAGE TRACKING SYSTEM"** submitted in partial fulfilment to the University as the project work of Bachelor of Engineering (Computer Science and Engineering) Degree, is a record of original work done by us under the supervision and guidance of **MR. M. KARTHICK RAJA M.E.**, Assistant Professor, Department of Computer Science and Engineering, Sri Eshwar College of Engineering, Coimbatore.

Place: Coimbatore RAGAV C S

Date: 722825104179

Project Guided by,

[MR. M. KARTHICK RAJA M.E., AP/CSE]

ACKNOWLEDGEMENT

The success of a work depends on a team and cooperation. I take this opportunity to express my gratitude and thanks to everyone who helped me in my project. I would like to thank the management for the constant support provided by them to complete this project.

It is indeed our great honor bounded duty to thank our beloved **Chairman Mr. R. Mohanram**, for his academic interest shown towards the students.

We are indebted to our **Director Mr. R. Rajaram**, for motivating and providing us with all facilities.

I wish to express my sincere regards and deep sense of gratitude to **Dr. Sudha Mohanram, M.E, Ph.D. Principal**, for the excellent facilities and encouragement provided during the course of the study and project.

We are indebted to **Dr. R. SUBHA, M.E., Ph.D.** Head of **Computer Science and Engineering Department** for having permitted us to carry out this project and giving the complete freedom to utilize the resources of the department.

I express my sincere thanks to my mini project Coordinator **Mr. M. Karthick Raja M.E.**, Assistant Professor of Computer Science and Engineering Department for the valuable guidance and encouragement given, to us for this project.

I solemnly express our thanks to all the teaching and nonteaching staff of the Computer Science and Engineering Department, family and friends for their valuable support which inspired us to work on this project.

TABLE OF CONTENT

CONTENT	PAGE NO
ABSTRACT	1
LIST OF FIGURES	2
LIST OF TABLES	3
1. INTRODUCTION	
1.1. PROJECT OVERVIEW & OBJECTIVES	4
1.2. PROBLEM STATEMENT AND MOTIVATION	5
2. SYSTEM REQUIREMENTS & SPECIFICATION	
2.1. FUNCTIONAL REQUIREMENTS	6
2.2. NON-FUNCTIONAL REQUIREMENTS	8
3. LOW-LEVEL DESIGN (LLD)	
3.1. SYSTEM ARCHITECTURE DESIGN	9
3.2. COMPONENT DESIGN & MODULE STRUCTURE	10
4. MODELING (UML DIAGRAMS)	
4.1. USE CASE DIAGRAM	12
4.2. CLASS DIAGRAM AND SEQUENCE DIAGRAM	13
4.3. FLOW DIAGRAM AND ACTIVITY DIAGRAM	14
5. DATABASE DESIGN	
5.1. ENTITY RELATIONSHIP DIAGRAM	16
5.2. DATABASE SCHEMA AND TABLE STRUCTURE	17
6. IMPLEMENTATION AND TESTING	
6.1. FILES WITH CODES	19
6.2. RESULT	20
7. CONCLUSION AND FUTURE SCOPE	
7.1. PROJECT SUMMARY AND ACHIEVEMENTS	21
7.2. FUTURE ENHANCEMENTS AND SCOPE	22
8. APPENDICES AND REFERENCES	
8.1. REFERENCES	23
8.2. APPENDIX A – SOURCE CODE STRUCTURE	23

ABSTRACT

The Smart Port & Airport Baggage Tracking System is a mission-critical web application developed to monitor, trace, and audit the movement of passenger luggage and container cargo across maritime port berths and international airport terminals. The system handles baggage check-in, tracking across multiple checkpoints, automated misplaced luggage detection, and handling fee invoices.

Developed using Java Spring Boot, the system manages baggage records, airline profiles, checkpoint scans, and dynamic fee calculations. Ground operators can register baggage at entry, generate a unique barcode tag, and complete scans at various transit points. The system also supports automated background checks for misplaced luggage using scheduled SQL operations.

In addition to operations, the system incorporates basic invoicing and carrier management functionalities. It maintains handling fee invoices, tracks payments, and generates operational reports. Facility administrators can monitor baggage status distributions and track outstanding invoice balances, providing a consolidated view of the terminal's operational and financial performance.

LIST OF FIGURES

Fig No	NAME OF THE FIGURE	Page No
3.1	SYSTEM ARCHITECTURE	9
3.2.1	MODULE STRUCTURE	10
3.2.2	COMPONENT DESIGN	11
4.1	USE CASE DIAGRAM	12
4.2.1	CLASS DIAGRAM	13
4.2.2	SEQUENCE DIAGRAM	13
4.3.1	FLOW DIAGRAM	14
4.3.2	ACTIVITY DIAGRAM	15
5.1	ER DIAGRAM	16
5.2	DATABASE SCHEMA	17

LIST OF TABLES

Table. No	NAME OF THE TABLE	Page No
5.2.1	AIRLINES TABLE	17
5.2.2	BAGGAGE TABLE	17
5.2.3	BAGGAGE_SCANS TABLE	18
5.2.4	INVOICES TABLE	18

INTRODUCTION

1.1 PROJECT OVERVIEW AND OBJECTIVES

The Smart Port & Airport Baggage Tracking System is a web-based management system designed to efficiently manage automated baggage tracking in port terminals. The system handles baggage tags, vehicle/flight entry and exit, transit duration, handling fee calculation, airline management, invoices, and terminal operations.

The application is developed using Spring Boot, MySQL, and Thymeleaf. It also integrates scheduled automation to manage misplaced baggage detection, carrier transactions, and operational metrics. The system provides operational reports to help administrators monitor the overall performance of the terminal handling facility.

Objectives

- To automate the management of baggage tracking and lifecycle sessions.
- To record checkpoint scans and automatically flag misplaced luggage.
- To manage airlines, carriers, tracking services, and related transactions.
- To automate invoicing and fee tracking operations.
- To track handling revenue, airline invoices, and terminal operations.
- To generate comprehensive operational reports for effective facility management.

1.2 PROBLEM STATEMENT AND MOTIVATION

Airports and seaports face a growing challenge of misplaced luggage, leading to inefficient manual searches, delayed transfers, and difficulties in managing handling operations. Traditional tracking systems often lack centralized tracking of transit duration, billing, carrier records, and automatic anomaly detection. This creates challenges in maintaining accurate records and monitoring the operational performance of a terminal facility. Manual processes can also result in billing errors, misplaced records, and delays in tracking. The lack of real-time information makes it difficult to identify delayed baggage and manage facility operations efficiently. A unified system is therefore required to connect tracking operations with automated alerts and reporting.

The project is motivated by the need for a centralized and automated tracking management solution that can simplify day-to-day operations. By combining baggage management with automated SQL checks and billing tracking, the system aims to reduce manual effort, improve record accuracy, and provide administrators with a clear view of both tracking operations and invoice performance. The system also provides a structured way to manage airlines, invoices, payments, and tracking activities. This integration can help terminal facilities operate more efficiently while supporting better planning and decision-making. The project also provides an opportunity to apply web technologies to a practical real-world management problem.

SYSTEM REQUIREMENTS & SPECIFICATION

2.1 FUNCTIONAL REQUIREMENTS

The system shall provide the following functional requirements:

Operator & Role Management

- Manage Terminal Operators and Administrative accounts.
- Provide restricted access to sensitive financial information.
- Maintain user information and activity logs implicitly.

Baggage Tracking Management

- Add, update, and manage unique baggage tags.
- Track status changes (CHECKED_IN, IN_TRANSIT, MISPLACED).
- Log physical checkpoint scan locations and times.

Automated Processing

- Run background tasks to detect overdue transit times.
- Flag baggage without scans for over 4 hours as MISPLACED.
- Calculate aggregated handling totals automatically.

Invoice & Airline Management

- Register and maintain carrier and airline profiles.
- Generate invoices for handling and terminal fees.
- Record invoice payments and track due dates.
- Compare paid amounts against pending balances.

2.2 NON-FUNCTIONAL REQUIREMENTS

The system shall provide the following non-functional requirements:

Performance

- The system should provide quick responses for common operations using JDBC.
- Baggage registration and search operations should be processed efficiently.
- Reports should be generated within a reasonable time using SQL aggregations.

Security

- SQL queries must use parameterized statements to prevent injection.
- Sensitive passenger data and financial information should be protected.
- Application configuration should securely store database credentials.

Reliability

- The system should maintain accurate tracking and financial records.
- Transactions should be stored consistently in the MySQL database.
- The system should handle invalid inputs gracefully with Jakarta Validation.

Usability

- The web interface should be simple and easy to navigate with Bootstrap/CSS.
- Forms should provide clear instructions and error messages.
- Operators should be able to access tracking history easily.

Maintainability

- The application should follow a modular Spring MVC architecture.
- Code should be organized across Controller, Service, and Repository layers.
- The MySQL database should maintain structured normalized schema.

LOW LEVEL DESIGN

3.1 SYSTEM ARCHITECTURE DESIGN

```
                    [PRESENTATION LAYER]
          Thymeleaf Templates | HTML5/CSS3 | Dashboard

                              ↓

                    [BUSINESS LOGIC LAYER]
      BaggageController | ScanController | InvoiceController
          BaggageService | ScanService | InvoiceService

                              ↓

                    [DATA ACCESS LAYER]
              Spring JDBC Template | RowMappers
            BaggageRepository | InvoiceRepository

                              ↓

                    [DATABASE LAYER]
                        MySQL 8.0
     baggage_db (tables: baggage, airlines, scans, invoices)
```

3.1 System Architecture

3.2 COMPONENT DESIGN AND MODULE STRUCTURE

```
src/main/java/com/example/usecaseprj/
├── controller/
│   ├── BaggageController.java
│   ├── ScanController.java
│   └── InvoiceController.java
├── model/
```

```
│ ├── Baggage.java
│ └── Invoice.java
├── repository/
│ ├── BaggageRepository.java
│ └── InvoiceRepository.java
└── service/
├── BaggageService.java
└── InvoiceService.java
```

3.2.1 Module Structure

IMPLEMENTATION AND RESULT

6.1 FILES WITH CODE

CONTROLLER

```
@Controller
@RequestMapping("/baggage")
public class BaggageController {
    // Controller logic for rendering tracking dashboard
}
```

ENTITY

```
public class Baggage {
    private String tagNumber;
    private String status;
    // Getters and setters
}
```

SERVICE

```
@Service
public class BaggageService {
    @Scheduled(fixedRate = 60000)
    public void flagMisplacedBaggage() { ... }
}
```

REPOSITORY

```
@Repository
public class BaggageRepository {
    private final JdbcTemplate jdbcTemplate;
    // SQL operations using jdbcTemplate
}
```

6.2 RESULT

The application successfully compiles and runs on Tomcat port 8080. The web interface correctly handles all CRUD operations and the automated background scanner correctly flags overdue baggage in the MySQL database.

19-20

CONCLUSION AND FUTURE SCOPE

7.1 PROJECT SUMMARY AND ACHIEVEMENTS

The Smart Port & Airport Baggage Tracking System is a web-based application developed to provide an organized and efficient solution for managing baggage in transport terminals. Built using Spring Boot, MySQL, and Thymeleaf, the system centralizes tracking operations including scan events, transit duration, automated anomaly detection, invoice calculation, and airline management.

ACHIEVEMENTS

- Developed a centralized system for automated terminal tracking.
- Implemented baggage lifecycle management with pure JDBC template SQL.
- Integrated automated background tasks to flag misplaced luggage.
- Designed a structured MySQL database with appropriate entities.
- Developed a layered application architecture.

7.2 FUTURE SCOPE AND ENHANCEMENTS

- **RFID Integration** – Connect the system with actual RFID readers for automated checkpoint scanning.
- **Mobile Application** – Develop a companion app for passengers to track their luggage in real time.
- **Payment Gateway** – Integrate digital payments for airlines to settle handling invoices directly.
- **Analytics Dashboard** – Add advanced visual analytics for operational bottlenecks.

APPENDICES AND REFERENCES

8.1 REFERENCES

- Spring Boot Documentation – Spring Reference Documentation.
- Spring Framework Documentation – Spring Web and JDBC.
- Thymeleaf Documentation – Thymeleaf User Guide.
- MySQL Documentation – MySQL 8.0 Reference Manual.
- Oracle Java Documentation – Java 21 Platform Reference.
- MDN Web Docs – HTML and CSS concepts.

8.2 APPENDIX A – SOURCE CODE STRUCTURE

```
usecaseprj/
│
├── src/main/java/com/example/usecaseprj/
│ │
│ ├── controller/
│ │ ├── BaggageController.java
│ │ ├── ScanController.java
│ │ ├── InvoiceController.java
│ │ └── ReportController.java
│ │
│ ├── model/
│ │ ├── Baggage.java
│ │ └── BaggageScan.java
│ │
│ ├── repository/
│ │ └── BaggageRepository.java
│ │
│ ├── service/
│ │ └── BaggageService.java
│ │
│ └── UsecaseprjApplication.java
│
└── src/main/resources/
├── templates/
├── application.properties
└── schema.sql
```